

From Engagement Artifact to Platform Asset

To: Engagement delivery team and analytics leadership | **Date:** 2026-09-22
Subject: What it would take to run this as a reusable capability rather than a one-off

This note is not part of the two required memos. It is the conversation I would want to have the week after this work shipped, because the interesting question is not whether the Cardinal number is right. It is whether the second, fifth and twentieth target cost us less effort than the first, and whether the answer gets measurably better each time.

1. The asset is not the model

What was built here reads as two scripts for two companies. It is actually three things, and only the third compounds.

A census onboarding pipeline. Every platform investment and every add-on arrives with a payroll export in a different shape. Turning that into a position-level model at fully loaded cost, reconciled to the P&L, is the same work every time, and once the crosswalks for a client exist the marginal run is free. That is the repeatable unit.

An outside-in estimator. Every pre-LOI target has no data room and the same public surface: a press release, an alternative-data extract, a fleet or footprint signal, a revenue figure. The method is portable; only the comp changes.

A structural comp library. This is the one that matters. Great Lakes was the comp for Cardinal *because Cardinal had already been through the onboarding pipeline in a prior deal*. When Cardinal closes and its real census lands, Cardinal becomes a comp for the next target. The library is the only part of this that gets more valuable with use, and it is the part that does not exist yet — today it is four operating companies living inside one repository.

2. What breaks first

An honest audit. Some of this is already production-shaped and should be said plainly, because knowing what *not* to rebuild is half of the judgement.

Already right. Mappings and assumptions are data, not code — `config/` holds every crosswalk, load-factor source, staleness cutoff and signal weight, each with a source and a rationale. Every imputation is logged to `output/exceptions.csv` with the rule that produced it. Boundaries raise rather than coerce: an unmapped job title stops the build with the exact pairs to add. Twenty-seven validation checks fail the run loudly. Outputs are deterministic. The reconciliation is a control, not a presentation layer.

Breaks immediately on a second client. The pipeline is 2,252 lines across four scripts with no test suite, and the client is baked in at several depths:

Where	What is hard-coded	Why it matters
<code>build_model.py:23-24</code>	<code>CENSUS_AS_OF</code> , <code>PNL_PERIOD_END</code> as module constants	Effective dates are per-engagement, not per-codebase
<code>build_model.py:18</code>	<code>DATA = ROOT / "candidate-exercise"</code>	One input location, one client
<code>build_cardinal.py:643</code>	<code>opco = "Cardinal Comfort Services"</code>	Target name written into the output
<code>backtest_structure.py:31</code>	<code>CORPORATE_OPKO = "Northstar Corporate"</code>	Shared-services detection is name-matched
<code>build_cardinal.py:28-35</code>	<code>FIELD_FAMILIES</code> , <code>GA_FAMILIES</code> , the three trade role constants	The taxonomy is a tuple in code

The last row is the deepest problem and worth being concrete about. The Cardinal estimator knows about HVAC service, HVAC install and plumbing service because the target is an HVAC and plumbing contractor. **Redwood Electric is already in the portfolio and has none of those roles** — its

field organisation is 99 Electricians and 90 Electrical Installers, and the string HVAC does not appear in it. Point this pipeline at an electrical add-on and it does not produce a wrong answer, it produces no answer. The role taxonomy has to become a versioned reference table with trades as data, not a set of constants, and the per-client crosswalks have to key into it rather than duplicate it.

The crosswalks themselves are per-client by construction — they key on (raw_department, raw_job_title), which is exactly right, but it means a canonical layer has to sit above them or every engagement reinvents the ontology.

3. Target shape, and what I would refuse to build

The failure mode here is over-engineering, so the anti-scope comes first.

No warehouse, no Spark, no orchestrator, no ML. The largest input is 1,665 rows and the pipeline runs in about a second. The cadence is per-deal, not per-hour. A gradient-boosted headcount model trained on four companies would be less accurate than the structural method and impossible to defend across a table from a CFO, which is the actual use. Auditable arithmetic is the product. No dashboard until there are at least three recurring consumers who are not analysts, because until then a CSV and a memo are strictly better.

What I would build. An installable package with a CLI, so an engagement is a command and a config directory rather than a branch:

```
workforce onboard --client greatlakes --as-of 2026-08-31
workforce estimate --target cardinal --comp greatlakes --as-of 2026-09-01
workforce backtest --library ./comps
```

Four changes carry most of the value:

- **A typed input contract.** data_dictionary.md is already the specification; encoding it as a schema means a malformed census fails at ingest with a readable column-level error instead of surfacing as a strange number three stages later.
- **Golden-master regression tests.** Freeze the Northstar outputs and fail CI on any unexplained change. With 2,252 lines and no tests, this is the single thing standing between the current code and safe refactoring, and it is roughly a day of work.
- **Run manifests and versioned output.** Write results to runs/{client}/{as_of}/ alongside a manifest carrying the git SHA, config hash and input file hashes. In diligence, *which version of the model did we price this on* is a question that gets asked, and it should have a one-command answer.
- **A reference layer above the crosswalks.** Canonical role, level and trade as versioned data with an owner; per-client mapping files key into it.

4. The flywheel: a comp library and a prediction ledger

Two mechanisms turn a pipeline into an asset that improves on its own.

The comp library. Every company that goes through onboarding contributes one row of structure: field share, level mix, span of control, load factors by department, part-time share by family, revenue per FTE. Anonymised and aggregated, that is proprietary benchmark data at position grain covering the acquirer's own portfolio — which no vendor sells, because no vendor has it. The back-test harness already written recomputes the method's error automatically as the library grows. Today it reports 2.5% mean absolute error on total field headcount from a comp set of four, with family-level error running 3.8% on Service, 7.3% on Install and 9.5% on Leadership. At fifteen companies that split should tighten materially, and the point is not that it will, it is that **we would know by how much without anyone deciding to check.**

The prediction ledger. This is the higher-leverage of the two and costs almost nothing. Every outside-in estimate is a falsifiable prediction with a natural resolution date: the close. When the deal closes, the real census flows through the onboarding pipeline and becomes ground truth for an estimate made months earlier. Log every prediction with its assumptions; score it on close. After ten or fifteen deals, the confidence bands in the Cardinal memo — $\pm 8\%$ on headcount,

±15% on cost — stop being my judgement and become measured calibration. Every deal is a free labelled example, and right now we throw all of them away.

It also creates the right institutional habit. A team that writes down what it expects and checks afterwards develops different instincts from one that only produces estimates.

5. Sequencing, ownership, and knowing it worked

Phase	Effort	Delivers
0. Package, golden tests, input schema	~2 days	Safe to refactor; a second client stops being a fork
1. Multi-client config, CLI, run manifests	~2 weeks	Any analyst can run an engagement unsupervised
2. Reference taxonomy, comp library, scheduled back-test	~1 quarter	Error rate becomes a tracked metric; Redwood-style trades supported
3. Prediction ledger and calibration	Ongoing	Confidence intervals become measured rather than asserted

Phase 0 is deliberately small and unblocks everything else. I would not start Phase 2 before a third company has been onboarded, because a library of two is a pair of anecdotes.

Ownership has to be named or the taxonomy rots. One person in analytics owns the canonical role ontology and reviews every new title mapping — the crosswalk validator already forces those to surface rather than default silently, which makes the review tractable. The pipeline needs a code owner for CI and releases. And the assumption register should be signed off by the deal lead before it prices anything, because `cardinal_assumptions.csv` contains at least one judgement — the 1.5x standalone G&A uplift — that moves the answer by more than a million dollars and should not be a modelling choice made quietly by an analyst.

Three metrics say whether this worked. Time from receiving a census to a reconciled model, which should fall from days to under one. Back-test mean absolute error, which should fall as the comp library grows. And prediction-ledger calibration: whether the stated ±8% band actually contains the truth about 8 times in 10. The third is the one that would make this a capability the deal team trusts by default rather than a model they ask someone to re-check.